

IGOT SIH — Antigravity Frontend Engineering Guide

Use this document after the Lovable UI is established. Preserve the design while connecting the application to real backend and prototype services.

1. Role of Antigravity

Take the existing Lovable-generated frontend as the visual baseline and turn it into a robust, maintainable, working application. Preserve the established visual identity unless a change is required for usability, responsiveness, accessibility, or engineering correctness.

Your job is to connect the frontend to the application's backend, database, AI/competency logic, and Mock iGOT API while keeping clear separation of concerns.

2. Existing Architecture

```
Frontend: Next.js / React + TypeScript
Backend: Next.js server/API business logic
Database/Auth/Storage: Supabase
AI services: competency analysis + recommendations
Assessment: quiz / reassessment / scoring
Integration: iGOT Adapter → Mock iGOT API
```

The official iGOT API is not assumed to be available for the prototype. Keep the integration replaceable so an authorized official iGOT API can be connected later.

3. First Task: Audit Before Editing

- Inspect the entire existing project before making architectural changes.
- Identify routes, components, services, hooks, API calls, types, database utilities, authentication, and mock data.
- Identify which parts came from the Lovable UI and preserve their visual language.
- Find duplicated components and consolidate only where it improves maintainability.
- Do not rewrite working UI unnecessarily.

4. Frontend Engineering Priorities

Area	Requirement	Expected Result
Routing	Organize major learner routes	Predictable navigation and protected pages
Auth	Integrate Supabase authentication	Authenticated learner experience
Data fetching	Use service/API layer	Components do not directly own backend logic
State	Use appropriate local/server state	No unnecessary global state
Forms	Validate inputs	Clear error and success states
Charts	Use real application data	Competency/progress visualizations update from data
Responsive UI	Test major breakpoints	No broken layouts on mobile/tablet

Area	Requirement	Expected Result
Errors	Handle API/database failures	Useful user-facing recovery states
Performance	Avoid unnecessary requests/renders	Fast dashboard and catalogue interactions

5. Mock iGOT Integration

Treat the Mock iGOT API as an external integration, not as frontend-only sample data.

```
Frontend
↓
Application service
↓
iGOT Adapter
↓
Mock iGOT API
```

- Implement a stable adapter interface.
- Keep course catalogue, course details, competency mapping, enrollment, progress, and completion operations behind that adapter.
- Do not expose secrets or integration credentials in the browser.
- Keep response types explicit with TypeScript interfaces.
- Make it possible to replace MockIGOTService with an OfficialIGOTService later without changing page components.

6. Suggested iGOT Adapter Contract

```
searchCourses(query, filters)
getCourse(courseId)
getCompetencies()
getCourseCompetencies(courseId)
enrollUser(userId, courseId)
getUserProgress(userId)
getUserCompletions(userId)
```

Only implement operations required by the current prototype. Do not invent undocumented official iGOT endpoints. These are internal application contracts for the mock integration.

7. Connect the Core Learning Flow

```
Login
↓
Dashboard
↓
Competency Profile
↓
Skill Gap Analysis
↓
AI Recommendations
↓
Mock iGOT Course
↓
Enrollment / Learning State
↓
Quiz
↓
Reassessment
```



The demo should feel like one continuous learner journey. A change in assessment/reassessment data should be reflected in competency and recommendation views where the backend supports it.

8. AI / Competency Integration

- Do not put AI prompts or private API keys in client-side code.
- Call AI services through secure server-side routes or backend services.
- Represent competency scores, gaps, evidence, recommendations, and explanations using typed data.
- Recommendation cards should show the competency gap addressed, recommendation reason, and course metadata.
- Keep scoring/recommendation logic out of UI components.

9. Database Integration

Use Supabase for persistent application data. Keep database access in server-side utilities/services where appropriate.

- Users / learner profiles
- Competency profiles
- Assessment attempts
- Quiz questions and answers
- Recommendations
- Course/enrollment state
- Learning progress
- Reassessment results

Use the existing schema if one already exists. Do not create duplicate tables simply because a UI screen needs a field.

10. Quality Checklist

- No broken routes or dead buttons.
- No hardcoded learner-specific results in production components.
- No secrets in client-side code.
- All API calls have loading and error handling.
- Forms validate correctly.
- Empty states exist for new learners and users with no recommendations.
- Dashboard data is consistent with backend responses.
- Responsive behavior is verified across desktop, tablet, and mobile.
- TypeScript errors are resolved.
- Console warnings/errors are resolved where practical.
- Mock iGOT calls are easy to identify and replace.

11. SIH Demo Readiness

Demo path:

Login → Dashboard → Competency Gap → AI Recommendation → iGOT Course → Quiz → Reassessment → Improved Competency → Analytics

Optimize this path for a reliable live demonstration. Seed a small, coherent dataset so the journey can be completed without manual database editing during the demo.

12. Final Instruction to Antigravity

Start by auditing the existing Lovable frontend. Preserve its design system and user experience. Then progressively connect it to Supabase, backend APIs, competency/AI services, quiz/reassessment logic, and the Mock iGOT Adapter. Keep business logic out of presentational components, keep iGOT integration modular, handle loading/error/empty states, and test the complete learner journey before making cosmetic changes.